

AI協助加速求解 12×12 十一塊拼圖實驗

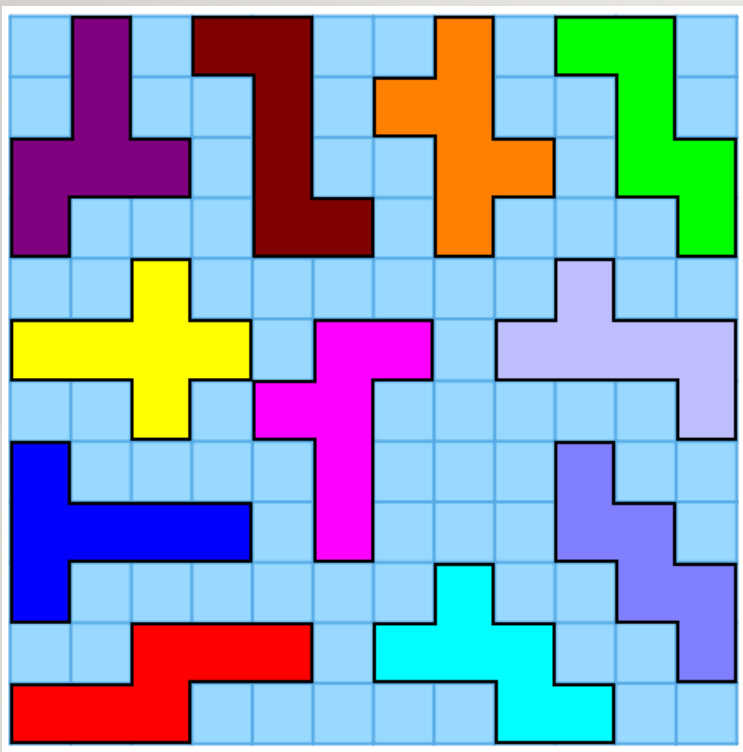
吳展宇

Untouchable 11

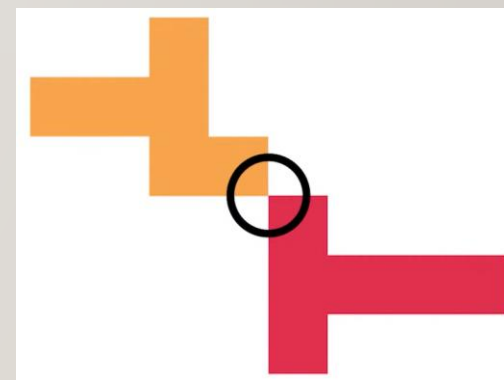
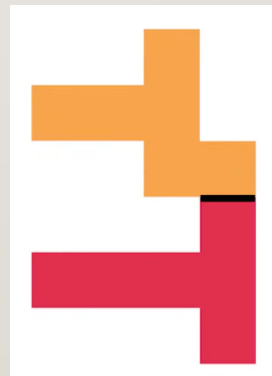
參考資料：

<https://www.youtube.com/watch?v=m6D5frpRSJ8>

<https://www.gathering4gardeners.org/g4g13gift/puzzles/HoffCarl-GiftExchange-FromUntouchable11toHazardCargo-G4G13.pdf>



有11塊不同的拼圖塊，每一塊都有六格，棋盤大小為12*12，目標要將所有拼圖塊都放進棋盤裡，每一個拼圖塊都可以旋轉、翻面，但任兩塊拼圖塊都不能有任何接觸，不管是邊還是角。



原始CNF子句建立方式

- **Each-piece-once (每塊恰好放一次)**：對每塊拼圖，其所有合法放法的變數中恰好一個為真——至少一個 (大 OR 子句) 加上兩兩互斥
- **No-overlap (不重疊)**：棋盤上每一格，最多只能被一塊拼圖佔用——對覆蓋同一格的所有 placement 變數兩兩加上 $\neg v_i \vee \neg v_j$ 。
- **No-touch (不接觸)**：任兩塊拼圖不得 8-鄰相接——對每一對 placement，若它們的覆蓋格之間存在 8-鄰關係，加上 $\neg v_i \vee \neg v_j$ 。
- **對稱破除 (piece 0 限上半)**：piece 0 只允許放在棋盤上半部，消除棋盤上下翻轉所產生的等價解。

實驗方法：

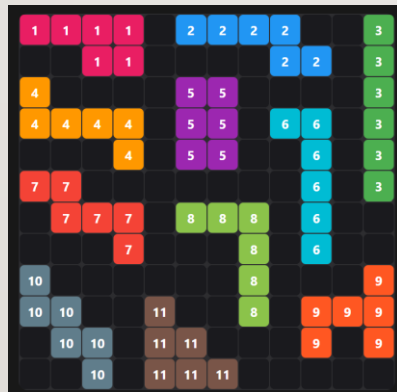
- 我自己隨便畫的新的拼圖，共有11塊六格大小的拼圖塊，並丟給ai請它想出基於原始 cnf 改良的方法(例如：加上新的**clause**或刪掉原有的**variable**)



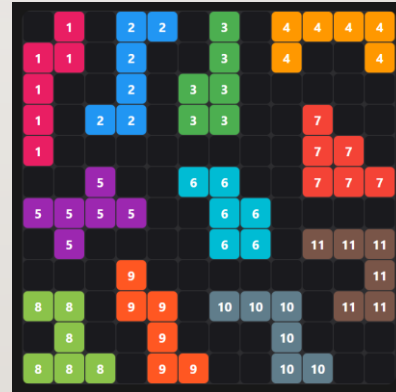
v1



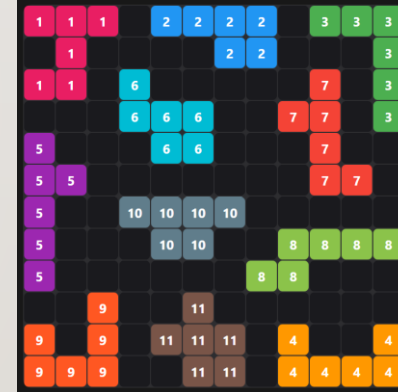
v2



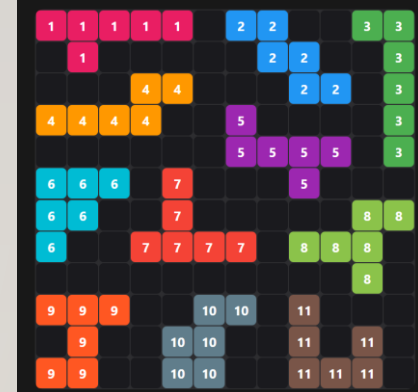
v3



v4



v5



v6

改良方法：shape

1. 邊角二元死區：

若兩塊不衝突的放置同時成立，卻在棋盤上留下 < 6 格的 4-連通空腔，則該空腔無法再容納任何六格塊（本題每塊固定 6 格）

→ 此組合比較不可能是完整 11 塊解的一部分，可預先加子句禁止。
(只考慮此兩塊裡至少有一格是在離邊緣四格以內的範圍)

	0	1	2	3	4	5	6	7
0	A	x	#	x	B	B	B	x
1	A	x	x	x	x	x	B	x
2	A	A	A	x		x	B	x
3	A	x	x	x		x	B	x
4	x	x				x	x	x

(0, 2) 的那格紅色格子就是一個死區

上次的v2公平枚舉實驗 (5 解 + D4 block)

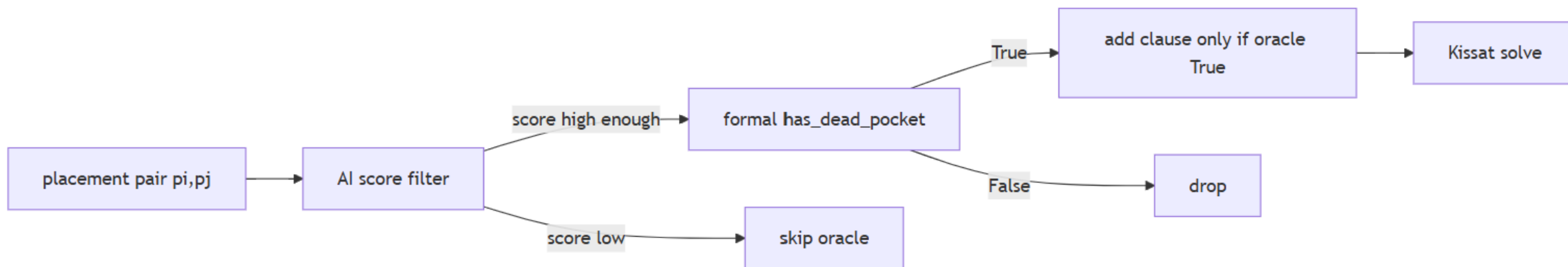
kissat seed = 0

方法	放置	變數*	子句	5 解平均 (s)	vs baseline	CNF 產生時間
baseline	6,120	12,229	4,420 萬	12.8	1x	26.5 s
shape	6,120	12,229	4,567 萬 (+147 萬)	9.7	1.32x	953.3 s
shape'	6,120	12,229	4,567 萬 (+147 萬)	11.3	1.13x	1056.2 s
shape''	5,864	11,717	4,818 萬 (+398 萬)	6.7	1.91x	3371.0 s

隱藏的問題：

1. baseline以外的方法 cnf產生的時間過久
2. 這個實驗方法不太公平(每個方法找出來的5個解都不同，解題方向不同，求解時間隨機性大)

改良方法一：加入ML RANKER



ML RANKER 訓練方式 (離線) :

- 資料：掃描 placement pair → 用 formal has_dead_pocket 自動標 0/1 (同 shape 幾何，不需人工)
- 特徵：v2 拼圖不變幾何特徵 (~78 維：bbox、邊距、shape sig、4×4 grid...)
- 模型：LightGBM LambdaRank (學「誰比較像 dead pocket」，不是單純分類)
- 目標：eval 拼圖上 Recall@25% 盡量高
- v123：在 v1+v2+v3 訓練 → v4 hold-out 測試

群組	維數	內容
A 單塊	16	bbox、質心、到邊距離
B 相對	12	row/col 重疊與間隙、禁區重疊、free_proxy、質心差、最近格距、bbox IoU、band 重疊長度
C 貼邊	6	margin=6 / margin=4 指示
D 形狀	24	6 格 canonical shape signature (旋轉翻轉不變)
E 分佈	16	4×4 粗格：空 / 禁區(0.5) / 佔據(1.0)
F 先驗	4	heur_score (= L0 公式) + 三項分解
合計	78	

方法一結果：

@25% recall：對全部候選對取 top 25% 進 oracle
表格中的數值：相對於shape的子句召回率

在v4上測試

訓練 / 模式	@25% recall	CNF 產生時間	@50% recall	CNF 產生時間
Baseline	X	37 s	X	X
v123 無 L0	42.1%	1195.5 s	70.7%	1761.3 s

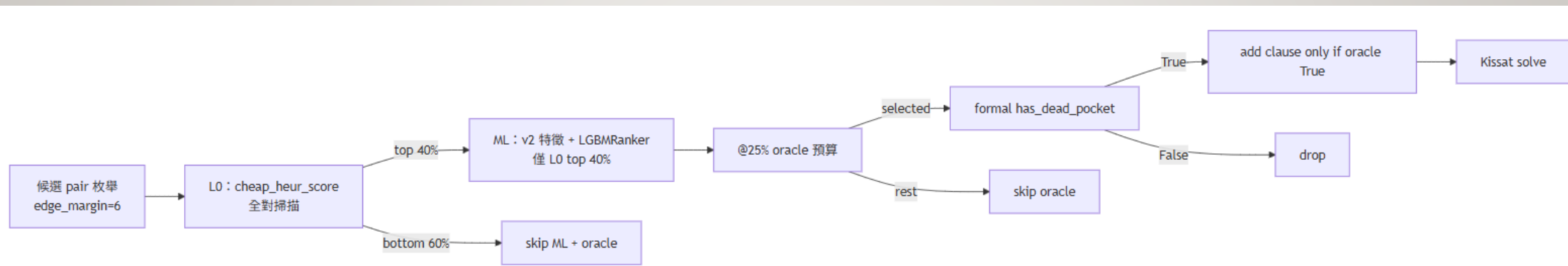
缺點：CNF 產生時間還是偏長

Shape的CNF 產生時間：2,437s(比較慢)

(10 seed Kissat)

模型	平均求解	vs baseline	vs shape
baseline	47.0 s	1.00×	—
shape	24.1 s	1.95×	1.00×
v123 無 L0@25%	28.3 s	1.66×	0.85× (慢於 shape)

改良方法二：在前面多加上極便宜的 L0 Filter



1. 先用極便宜的 L0 幾何啟發式砍掉大部分 pair
2. 再對存活對跑完整 ML ranker
3. 最後才進 oracle

$L0_score =$
 $0.4 / \max(\text{free_proxy}, 1)$
 $+ 0.35 \times [\text{任一方在邊緣 } 4 \text{ 格內}]$
 $+ 0.25 \times [\text{兩塊 bbox 在 row 方向重疊}]$

項	意義	直覺
0.4 / free_proxy	兩塊禁區合併後，剩餘可放格越少 → 分越高	越擠 → 越容易擠出小死腔
+0.35 貼邊	A 或 B 有格在離邊 ≤4 格	邊角是 shape 死區高發區
+0.25 同行	兩塊 bbox 在 row 方向重疊	同一水平帶互卡，易留狹縫

方法二結果：

(10 seed Kissat)

在v4上測試

訓練 / 模式	@25% recall	CNF 產生時間	@50% recall	CNF 產生時間
Baseline	—	37s	—	—
v123 無 L0	42.1%	1195.5 s	70.7%	1761.3 s
v123 L0	34.72%	999 s	53.34%	1233.3 s

模型	平均求解	vs baseline	vs shape
baseline	47.0 s	1.00×	—
shape	24.1 s	1.95×	1.00×
v123 無 L0@25%	28.3 s	1.66×	0.85× (慢於 shape)
v123 L0@25%	28.1 s	1.67×	0.86× (約等於 無L0)

v4 , v5 , v6結果比較：

(10 seed Kissat)

拼圖	cascade @25% recall	CNF 產生時間 Learned vs shape	SAT learned vs shape	SAT learned vs baseline
v4	34.72%	2.44× (快)	0.86× (慢)	1.67×
v5	34.21%	1.30× (快)	0.97× (接近)	1.61×
v6	33.84%	1.41× (快)	0.75× (慢)	1.16×

增加訓練資料集

在v6上測試

(10 seed Kissat)

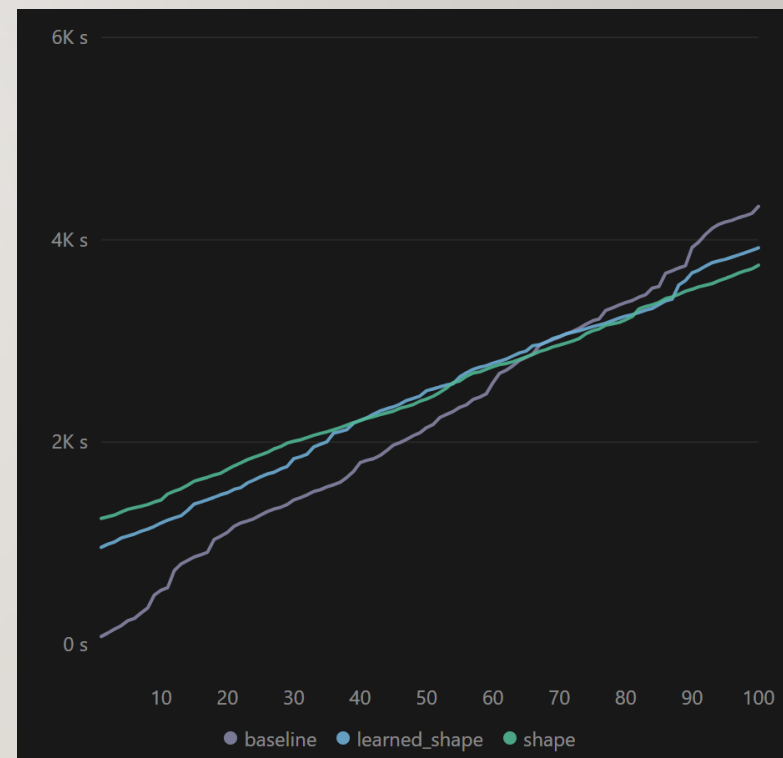
模型	CNF 產生時間	SAT mean	vs shape (32.3s)
v123 L0@25%	1,461 s	43.2 s	0.75× (慢)
v12345 L0@25%	951 s	31.2 s	1.04× (接近)

連續解100個相異解 比較總體時間：

在v6上測試

方法	CNF build	平均 solve	100 解累積	k* vs baseline
baseline	31.5 s	43.0 s	4330 s	—
v12345 L0@25%	938.3 s	29.8 s	3920 s	68
shape	1232.5 s	25.2 s	3749 s	66

(shape 於 k=40 起 總體累積時間贏 v12345 L0@25%)



結論：

- L0 + ML ranker可以花費比shape方法更短的時間產生cnf，並得到比shape略差一點的求解速度(還是比baseline方法的求解速度快)
- 雖然L0 + ML ranker的方法求解速度比baseline還快，但前置處理時間過長，在v6的連續解100個相異解實驗中可以發現大概需要連續求解68個相異解以上，此方法才能在總體花費的時間上超越baseline
- 但是shape方法於 $k=40$ 起總體花費時間就贏L0 + ML ranker的方法了 因此如果要解數十個相異解以上不如直接選shape方法就好了